Class: SE **Subject: COA LAB** **Sem-III**

Experiment No 2

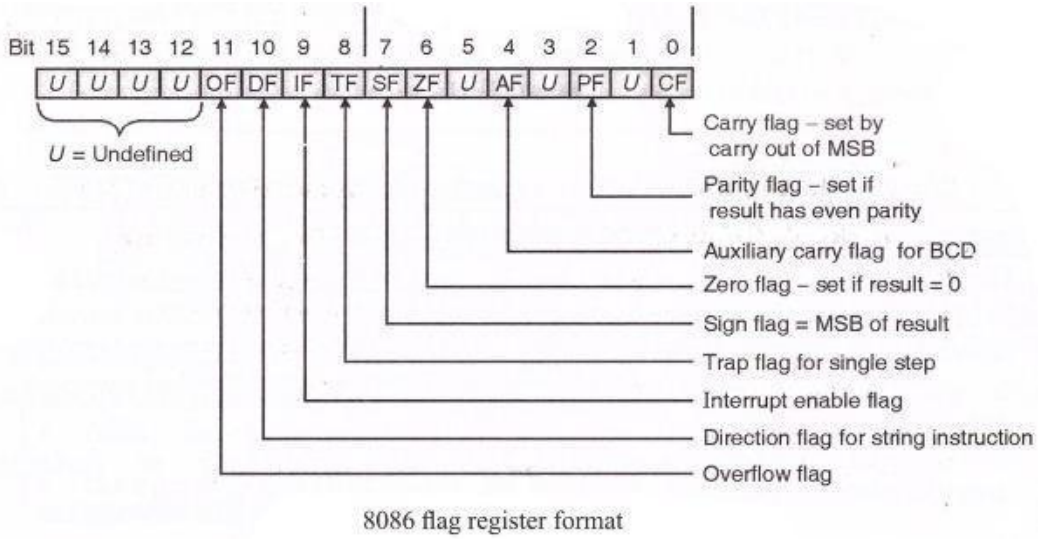
Course Outcome: CO1 **Blooms Level: L3**

Aim: To apply 8086 instruction set knowledge for evaluating and displaying the status of condition flags using flag manipulation instructions.

Abstract: Develop an assembly language program using the MASM assembler that analyzes and displays the status of individual flags in the 8086 processor's flag register. The program should begin by initializing the data segment and printing a formatted header that labels key processor flags such as OF, DF, IF, SF, ZF, AF, PF, and CF. It must then set and clear specific flags using instructions like STC, STD, and CLI. The flag register is expected to be pushed onto the stack and popped into a general-purpose register for bitwise evaluation. Using a rotating bitmask, the program needs to check each of the 16 flag bits from bit 15 to bit 0 and display their corresponding binary values (1 or 0) on the screen. DOS interrupts should be used for character output, and the results must be presented in a clear, readable format.

Theory:

Flag register is a part of the EU (Execution Unit). It is a 16 bit register with each bit corresponding to a flip-flop. A flag is a flip-flop. It indicates some condition produced by the execution of an instruction. For example the zero flag (ZF) will be set if the result of execution of an instruction is zero. Figure below shows the details of the 16 bit flag register of the 8086 CPU.



8086 flag register format

• It consists of 9 active flags out of 16. The remaining 7 flags marked 'U' are undefined flags.
• These 9 flags are of two types:
6 Status flags

3 Control flags

Status flags:

1. Carry flag (CY)-
• It is set whenever there is a carry or borrow out of the MSB (most significant bit) of a result. D7 bit for an 8 bit operation and D15 bit for a 16 bit operation.

2. Parity flag (PF)-
• It is set if the result has even parity. If parity is odd, PF is reset.
• This flag is normally used for data transmission errors.

3. Auxiliary carry flag (AC)-
• It is set if a carry is generated out of the lower nibble.
• It is used only in 8 bit operations like DAA and DAS.

4. Zero flag (ZF)-
• It is set if the result is zero.

5. Sign flag (SF)-
• It is set if the MSB of the result is 1. For signed operations such a number is treated as negative.

6. Overflow flag (OF)-
• It will be set if the result of a signed operation is too large to fit in the number of bits available to represent it.
• It can be checked using the instruction INTO (Interrupt on Overflow).

Control flags:

1. Trap flag (TF)-
• It is used to set the trace mode i.e. start single stepping mode.
• Here the microprocessor is interrupted after every instruction so that the program can be de-bugged.

2. Interrupt enable flag (IF)-
• It is used to mask (disable) or unmask (enable) the INTR interrupt.
• If the user sets the IF flag, the CPU will recognize external interrupt requests. Clearing IF disables these interrupts.

3. Direction flag (DF)-
• If this flag is set, SI and DI are in auto-decrementing mode in string operations.

Class: SE **Subject: COA LAB** **Sem-III**

Program:

```
.model small
.data
msg db 0dh, 0ah, "----------- OF DF IF TF SF ZF -- AF -- PF -- CF $"
new1 db 0dh, 0ah, "$"
flag dw ?
.code
start:
mov ax,@data
mov DS,ax

mov dx,offset msg
mov ah,09h
int 21h

mov dx,offset new1
mov ah,09h
int 21h

cli
stc
std

pushf
pop BX
mov flag,bx
mov cx,16
mov bx,8000h  ; 1000 0000 0000 0000

loops:
        mov ax,flag
        and ax,bx
        jz zero
        mov dl,31h
        mov ah,02h
        int 21h
        jmp space

zero:
        mov dl,30h
        mov ah,02h
        int 21h

space:
        mov dl,' '
        mov ah,02h
        int 21h

        mov ah,02h
        int 21h

        ror bx,1
        loop loops

mov ah,4ch
int 21h
end start
```

Output:

```
C:\>exp2.exe

----------- OF DF IF TF SF ZF -- AF -- PF -- CF
0  1  1  1  0  1  0  0  0  0  0  0  0  1  1
```

Class: SE **Subject: COA LAB** **Sem-III**

Conclusion: The experiment successfully demonstrated how to access, interpret, and display the status of various condition flags in the 8086 microprocessor. The flag register plays a crucial role in determining the outcome of arithmetic and logical operations, as it reflects the processor's current state through individual flag bits such as Carry (CF), Zero (ZF), Sign (SF), Overflow (OF), Parity (PF), and Auxiliary Carry (AF).By using instructions such as STC, STD, and CLI, specific flags are manually set or cleared, and the complete flag register is then captured and evaluated bit by bit using a rotating mask. The use of logical operations and DOS interrupts ensures that each flag's binary state is accurately printed in a structured format.

Exercise 1:

Aim: To perform 8-bit addition and evaluate the status of relevant condition flags.

Abstract: Design a program or perform manual flag evaluation for an 8-bit addition operation involving two operands: i)66h and 19h ii) FFh and 01h .

The task is to compute the result of the addition and determine the status of various condition flags in the processor after the operation. These flags include the Carry flag (CF),Auxiliary Carry Flag (AF), Zero flag (ZF), Overflow flag (OF), Sign flag (SF), and Parity flag (PF). The program or analysis should correctly reflect how these flags are set or cleared based on the result and intermediate bit patterns during the addition. This task aims to enhance understanding of flag behavior in arithmetic operations within an 8-bit architecture.